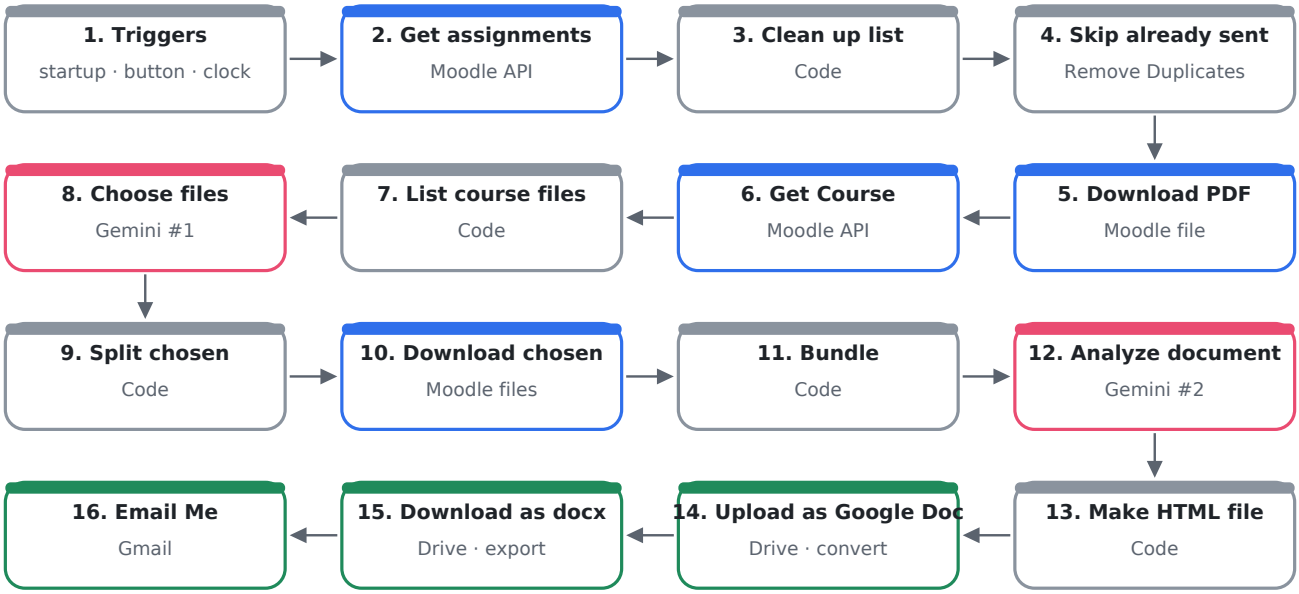
Homework Autopilot

A self-hosted n8n pipeline that signs into ZUJ's e-learning (Moodle) through its official API, finds new assignments, picks the relevant lecture files, and emails me a study guide with real slide citations.

What it does: Three times a day (and whenever n8n starts) the workflow asks Moodle for my assignments, ignores anything already handled, downloads the new assignment PDF, lists the course's lecture files, lets Gemini pick the 3–4 most relevant ones, then has Gemini write a per-question study guide that cites those files by page. Google Drive turns it into a Word document and Gmail delivers it. **Nothing is ever submitted.** If any step fails, a second workflow emails me an alert.

The whole pipeline (16 nodes)



Blue = talks to Moodle · Pink = AI (Gemini) · Green = Google Drive / Gmail · Grey = logic that runs inside n8n.

Contents

Part	Topic
1	The portal: how n8n gets into Moodle and what it fetches (the main part)
2	Docker: why n8n runs in a container and how it's set up
3	n8n basics: items, expressions, binary data, credentials
4	Every node explained, in order
5	AI, documents, delivery: Gemini, Drive conversion, Gmail
6	Reliability, security and cost
7	Limitations, roadmap and FAQ

PART 1

Getting into the portal

ZUJ's e-learning site is **Moodle**, the open-source learning platform. Besides the website you click around in, Moodle has a **Web Services API**: a machine-friendly door that returns clean JSON instead of web pages. The official Moodle mobile app uses exactly this door, which is why it can work at all. My workflow uses the same door, as the same kind of client.

Why not just scrape the website?

	Scraping the website	Moodle Web Services API
Login	Would need to automate Microsoft SSO + Authenticator: fragile and the kind of thing security systems flag	One token, issued by Moodle's own app-login feature
Data	HTML pages, so you have to parse layouts that change with every theme update	Structured JSON with stable field names
Scope	Full logged-in browser session	Only the functions the mobile service allows; revocable on its own
Looks like	A bot clicking through pages	An API client, the same category as the phone app

Step 0: Is the door even open?

Before anything else, I asked the site what it supports. Moodle exposes a **public, no-login** function that the mobile app calls first: `tool_mobile_get_public_config`. Anyone can call it; it reveals no personal data.

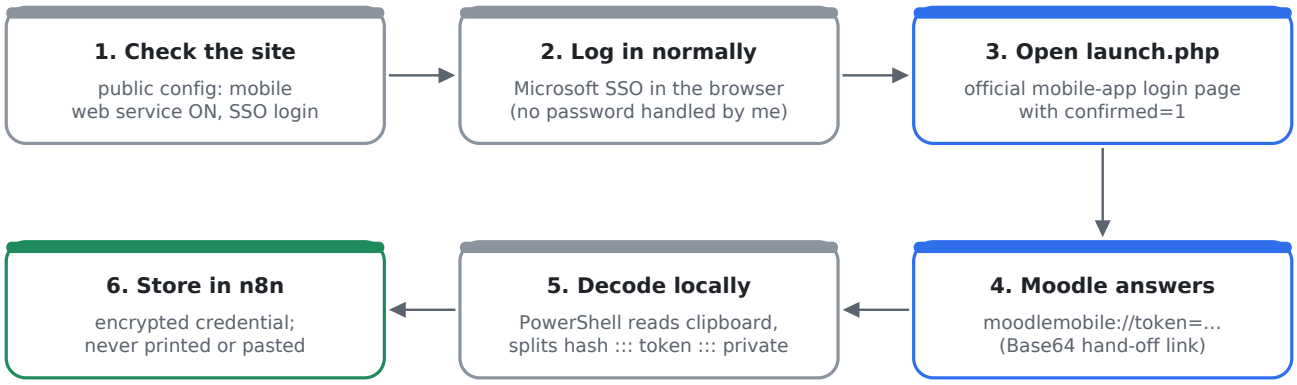
```
POST https://elearning.zuj.edu.jo/lib/ajax/service-nologin.php
[{"index":0,"methodname":"tool_mobile_get_public_config","args":{}}]

enablewebservices      : 1     <- the API exists
enablemobilewebservice : 1     <- students may use it via the mobile service
typeoflogin            : 2     <- login happens in a browser (SSO)
launchurl              : https://elearning.zuj.edu.jo/admin/tool/mobile/launch.php
```

The answer that made the project possible: the university has deliberately enabled the mobile web service.

Step 1: Getting a token when there's no password

The classic way to get a token is `login/token.php` with a username and password. ZUJ moved to **Microsoft single sign-on**, so my account has no Moodle password and that route fails. **typeoflogin = 2** says what the app does instead: open the login in a browser, then let Moodle hand a token back to the app. I did that hand-off myself:



```
https://elearning.zuj.edu.jo/admin/tool/mobile/launch.php
    ?service=moodle_mobile_app   <- which API service the token is for
    &passport=12345              <- random value the app uses to check the reply
    &urlscheme=moodlemobile      <- 'send the result to the Moodle app'
    &confirmed=1                 <- show a link instead of auto-redirecting
```

Without confirmed=1, Chrome silently swallowed the moodlemobile:// redirect (no app installed), so nothing appeared.

Decoding the hand-off

The link looks like `moodlemobile://token=YmNl…`. The part after `token=` is **Base64**, an encoding (not encryption). Decoded, it's three values separated by **:::**

Part	What it is	Used?
1st	A check value: MD5 hash of the site URL + the passport I sent, so the app can confirm the reply matches its request	No
2nd	**The web-service token**: 32 hex characters, the key the API accepts	Yes
3rd	A 'private token' the app uses to get auto-login links to the website. More sensitive.	Deliberately ignored

A small PowerShell function reads the link from the clipboard, decodes it, verifies it, and puts **only the token** back on the clipboard, so the token is never printed on screen or pasted into a chat.

```
function Get-MoodleToken {
  $link  = (Get-Clipboard | Out-String).Trim()
  $enc   = $link -replace '^moodlemobile://token=', ''
  $token = ([Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($enc)) -split ':::')[1]
  # verify: who am I, and is the function I need allowed?
  $r = Invoke-RestMethod -Method Post -Uri '.../webservice/rest/server.php' -Body @{
         wstoken=$token; wsfunction='core_webservice_get_site_info'; moodlewsrestformat='json' }
  $r.fullname                                                  # -> my name
  $r.functions.name -contains 'mod_assign_get_assignments' # -> True
  $token | Set-Clipboard
}
```

> **Why this is legit:** Microsoft login and MFA are never bypassed. I log in normally in my own browser; Moodle's official app-login page then issues a token for **my own account**, scoped to the mobile service. It's the same mechanism the phone app uses.

What the token is (and isn't)

It is **not a JWT**. A JWT carries its own data (claims) and a signature, so a server can trust it without a lookup. A Moodle token is **opaque**: a random string that means nothing alone. Moodle stores it in a database table (`external_tokens`) together with the user, the service, the creation time and an optional expiry, and looks it up on every request.

	JWT	Moodle token
Contains data?	Yes (decodable claims)	No, a random 32-char string
How it's checked	Verify signature	Database lookup
Revoke early	Hard (valid until expiry)	Easy: Preferences → Security keys → Reset deletes the row
Scope	Whatever the claims say	Only functions allowed by the moodle_mobile_app service

That's why each **Reset** during setup killed the old token instantly, and why the check printing **True** mattered: it proved the service includes the function I need.

Talking to the API: one endpoint, many functions

Every API call goes to the same URL. What changes is the `wsfunction` parameter. The token travels as `wstoken` in the query string, added automatically by an n8n credential so it never appears inside the workflow.

```
POST https://elearning.zuj.edu.jo/webservice/rest/server.php?wstoken=••••••••
Content-Type: application/x-www-form-urlencoded

wsfunction=mod_assign_get_assignments&moodlewsrestformat=json
```

Function	Where it's used	What comes back
`tool_mobile_get_public_config`	One-time check (no login)	Site capabilities: is the mobile API on, how login works
`core_webservice_get_site_info`	One-time check	Who the token belongs to + every function it may call
`mod_assign_get_assignments`	Node 2: Get assignments	All my courses, each with its assignments, due dates and attachments
`core_course_get_contents`	Node 6: Get Course	All sections of one course and every file in them

What the assignments reply looks like

```
{ "courses": [
   { "id": 5174,
     "fullname": "{mlang en}20261-Algorithms-AllSec-3-F{mlang}{mlang ar}...{mlang}",
     "assignments": [
       { "id": 9612, "name": "Phase 1",
         "duedate": 1794131940,              <- Unix time (seconds since 1970)
         "cutoffdate": 1794171540,
         "intro": "",                        <- instructions text (empty here)
         "introattachments": [
           { "filename": "Algorithms Course Project Phase 1.pdf",
             "fileurl": "https://elearning.zuj.edu.jo/webservice/pluginfile.php/.../Phase%201.pdf
",
             "mimetype": "application/pdf" } ] } ] },
   { "id": 5294, "fullname": "...Machine Learning...", "assignments": [] } ],
  "warnings": [ { "message": "No access rights in module context" }, ... ] }
```

The warnings are normal: activities hidden by the instructor or not yet open.

- `{mlang en}…{mlang}`: ZUJ stores English and Arabic names in one field; the Clean up list node keeps the English part.
- **Unix timestamps**: converted to readable Amman time (1794131940 → 08/11/2026, 12:59).
- **Two kinds of URL**: API calls use `wstoken`, but file downloads through `webservice/pluginfile.php` expect the same token named `token`. That's why the workflow has two credentials holding the same value under different names.

What the course-contents reply gave us

For Algorithms, `core_course_get_contents` returned **17 sections**, one per topic ("1: Efficiency" … "14: Dijkstra's Algorithm"), containing resources, pages, links and labels. After filtering to PDFs and removing a duplicate, about **25 lecture files (~22 MB)** remained: numbered slides like `08_Graphs.pdf` plus notes and worked examples. All shared the same upload date, so 'pick the newest' was useless, which led to letting Gemini choose (Part 5).

Can the university see this?

Yes, and it doesn't need to be hidden. Admins can see that a mobile-service token exists for my account, and Moodle can log each web-service call (marked with origin `ws`, like the phone app). Server logs record a generic HTTP-library client instead of the Moodle app. What they can't see is anything after the data leaves Moodle. The workflow keeps a polite footprint: a few requests, three times a day, for my own data, through the door the university opened.

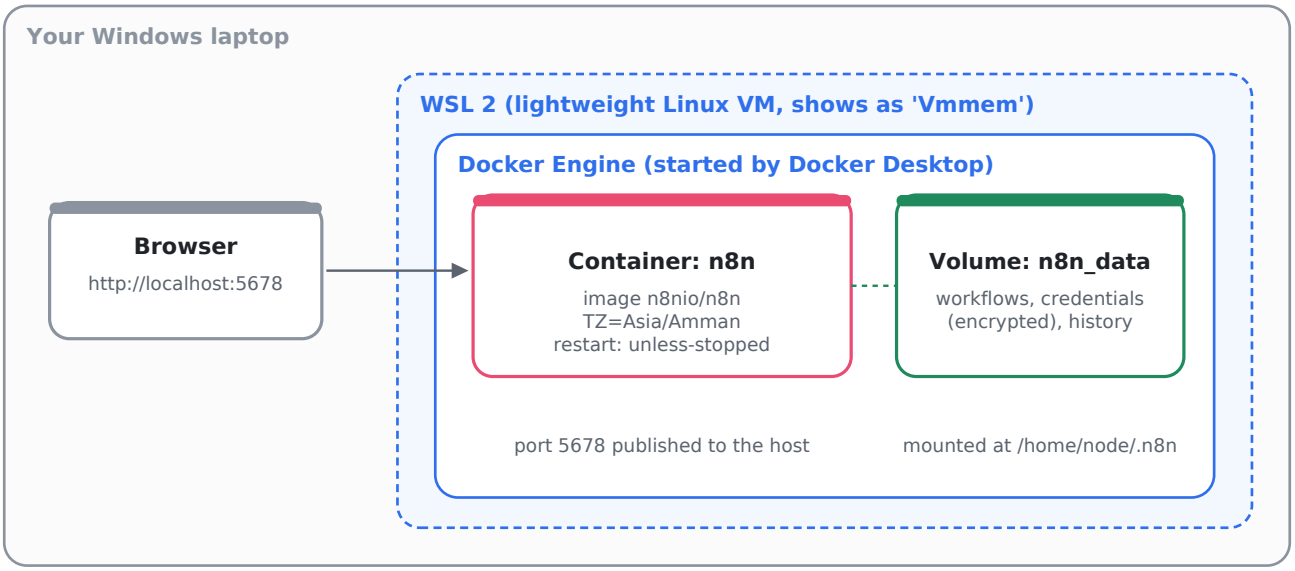
PART 2

Docker: where n8n lives

n8n is a Node.js application. I could install Node and n8n directly on Windows, but n8n's own docs recommend Docker, and it's the standard way to self-host server software.

Why Docker?

- **Isolation:** n8n and all its dependencies live in a sealed box (a **container**). Nothing is installed into Windows itself, and nothing on Windows can conflict with it.
- **Reproducible:** the container is created from an **image** (n8nio/n8n), a frozen, versioned snapshot of a working setup. Same image, same behaviour, on any machine, which matters when a friend sets it up too.
- **Disposable, but data persists:** containers can be deleted and recreated any time. Everything worth keeping lives in a **volume** that survives updates and restarts.
- **Easy updates and clean removal:** pull a new image to upgrade; delete the container and volume and it's completely gone.



On Windows, Docker Desktop runs a small Linux virtual machine through **WSL 2**; containers run inside it. That's why Task Manager shows 'Vmmem' using memory, and why `wsl --shutdown` frees it after quitting Docker Desktop.

The whole setup is one file

```
services:
  n8n:
    image: n8nio/n8n                # which image to run
    container_name: n8n
    restart: unless-stopped         # come back automatically when Docker starts
    ports:
      - "5678:5678"                 # host port : container port  -> http://localhost:5678
    environment:
      - GENERIC_TIMEZONE=Asia/Amman  # used by n8n's Schedule Trigger
      - TZ=Asia/Amman               # system clock inside the container
      - N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true
    volumes:
      - n8n_data:/home/node/.n8n    # persistent storage

volumes:
  n8n_data:
```

docker-compose.yml: Docker Compose turns this description into a running container.

Concept	In this project
Image	n8nio/n8n, downloaded once (~45 s on the first run)
Container	the running n8n instance, named n8n (~440 MB RAM, near-zero CPU when idle)
Port mapping	5678 inside → 5678 on Windows, so the editor opens at localhost:5678
Volume	n8n_data: workflows, encrypted credentials, execution history, the encryption key
Env vars	two timezone variables. Without GENERIC_TIMEZONE, '9:00' would mean 9:00 UTC (12:00 in Amman)
Restart policy	unless-stopped: starts with Docker Desktop, unless I stopped it on purpose

Everyday commands

```
docker compose up -d        # create/start n8n in the background
docker compose restart      # restart (also fires the 'On startup' trigger)
docker compose logs -f n8n  # watch logs when debugging
docker compose down         # stop and remove the container (data stays in the volume)
docker compose pull         # download a newer n8n image, then 'up -d' again
wsl --shutdown              # after quitting Docker Desktop: release the VM's memory
```

> **Trade-off:** Self-hosting means the workflow only runs while the laptop is on and Docker Desktop is running. Missed scheduled runs aren't replayed, but the next run catches anything new, and the 'On startup' trigger checks as soon as n8n boots. A cheap VPS would make it 24/7 with the same compose file.

PART 3

n8n in five ideas

Idea	What it means	Where you see it here
Nodes	Each box does one job: call an API, run code, send an email.	16 nodes in the main workflow, 2 in the error workflow
Items	Data flows as a list of JSON objects. A node runs once per incoming item automatically.	Download chosen gets 3 items → downloads 3 files, no loop needed
Expressions	Anything inside `{{ }}` is JavaScript evaluated at run time.	`{{ $json.files[0].url }}` picks the PDF link from the previous node
Binary data	Files travel next to the JSON under named slots (data, slide1, ...).	PDFs, the HTML page, the .docx attachment
Credentials	Secrets stored encrypted, attached to nodes by name, never exported with the workflow.	Moodle API token, Moodle file token, Gemini key, Google Drive, Gmail

Reaching back to earlier nodes

`$json` means 'the current item from the previous node'. `$('Clean up list').first().json.course` reaches back to a specific node by name. One lesson learned: `.item` tries to trace each item back to its origin, and that tracing breaks after a node that merges many items into one (Bundle), so the prompts use `.first()`. Another one: a field only evaluates expressions when it's switched to **Expression** mode. In Fixed mode, Gemini once received the literal text `{{ $json.menu }}` instead of the file list.

Publishing and executions

Editing happens on a draft; **Publish** makes it the live version that triggers use. Every run, manual or automatic, appears in the **Executions** tab, where a failed run can be opened on the canvas to see exactly which node failed and with what data.

PART 4

Every node, in order

#	Node	Type	What it does
1	Triggers	n8n Trigger · Manual · Schedule	On startup (instance started), a test button, and a clock at 09:00, 18:00 and 00:00 Amman time. All three feed node 2.
2	Get assignments	HTTP Request	POST `mod_assign_get_assignments`, with the API token as `wstoken`.
3	Clean up list	Code	Flattens courses → assignments, keeps the English course name, converts dates, skips past-due ones, outputs one item per open assignment (course, courseId, assignmentId, due, files).
4	Skip already sent	Remove Duplicates	'Remove items processed in previous executions' on assignmentId. n8n remembers IDs across runs, so each assignment is emailed once.
5	Download PDF	HTTP Request	GET `{{ $json.files[0].url }}` with the file token; response format = File.
6	Get Course	HTTP Request	POST `core_course_get_contents` for that courseId: every section and file.

#	Node	Type	What it does
7	List course files	Code	Keeps unique PDFs, numbers them into a short 'menu', and carries the assignment PDF along.
8	Choose files	Gemini (Analyze document)	Reads the assignment + menu, replies only with a JSON array like [9, 10, 12].
9	Split chosen	Code	Parses the array (tolerates ```json fences), maps numbers to files, caps at 4, falls back to file 1.
10	Download chosen	HTTP Request	Runs once per chosen file and downloads each PDF.
11	Bundle	Code	Packs assignment + slides into one item (assignment, slide1, slide2…) plus a numbered file list for the prompt.
12	Analyze document	Gemini (Analyze document)	Sends all files (`binaryNames`) with the study-guide prompt; returns HTML.
13	Make HTML file	Code	Strips stray fences, wraps the HTML in a UTF-8 page, builds the file name.
14	Upload as Google Doc	HTTP Request (Drive API)	Uploads with `convert=true`, and Google turns the HTML into a native Doc.
15	Download as docx	Google Drive	Exports that Doc as MS Word (real heading styles, real tables, superscripts kept).
16	Email Me	Gmail	Sends course, assignment, due date and the lecture files used, with the .docx attached.

PART 5

The AI part: retrieve, then generate

Sending a whole course (~22 MB of PDFs) to the model on every run would be slow and wasteful. So the workflow uses **two** Gemini calls, a small-scale version of the **RAG** pattern (retrieval-augmented generation):

	Call 1: Choose files	Call 2: Analyze document
Input	Assignment PDF + a text menu of ~25 file names	Assignment PDF + the 3–4 chosen lecture PDFs + their names
Job	Retrieval: which lectures matter?	Generation: write the study guide, citing pages
Output	[9, 10, 12]	Clean HTML, one section per question
Cost/time	Tiny and fast	The heavy call, kept small by step 1

The study-guide format

For each question or task in the assignment's own numbering: **Core Concept** (with a citation like `course_description.pdf, p. 2`, or 'Review: topic' when no attached file covers it), **Guided Logic Breakdown**, **Worked Solution and Check** (with 1–2 self-check questions), and a one-sentence **Key Takeaway**. Open-ended project tasks get 'how to approach it' guidance instead of a solution.

Prompt rules that do the quality control

- **Only cite pages actually seen** in the attached files; never invent lecture or slide numbers.
- **Never invent references, URLs, datasets or statistics.** An early run produced a realistic-looking fake IEEE citation tagged [VERIFY]. The rule was tightened so illustrative examples must be labeled **[EXAMPLE FORMAT ONLY – not a real paper]**, and the next run complied.
- **[TO FILL]** for anything only the student or team can decide (names, IDs, topic choice).
- **No LaTeX** (it doesn't render in Word): plain text or <sup>/<sub>, which survive the conversion as real superscripts.
- **HTML output**: structure that a converter can turn into proper Word formatting.

Model choice and the timeout lesson

The first full draft with the regular Flash model 'thought' for minutes and the connection was cut (n8n's HTTP calls default to a 5-minute limit). Switching to **Flash-Lite** plus **Retry On Fail** fixed reliability, at a cost: Flash-Lite is fast but shallow, so the guide is a map of what to study, not an answer key. A timed-out request returns nothing, so it can't be 'resumed'. The real fixes are splitting work into smaller calls (one per question) or calling the API through an HTTP Request node with a longer timeout.

Why HTML → Google Doc → .docx?

n8n has no built-in Word generator. Instead of building one, the workflow asks Gemini for HTML and lets **Google Drive's import converter** do the formatting: uploading with `convert=true` produces a native Google Doc, and Drive's export produces a .docx where headings are real Word heading styles (so the navigation pane and auto table of contents work) and tables are real tables. The same trick works in reverse: PowerPoint or Word lecture files could be converted to PDF for Gemini.

Delivery

Gmail sends the .docx to my own inbox with the course, assignment, due date, and the list of lecture files the citations came from, which is a quick way to judge whether the file choice was sensible.

PART 6

Reliability, security and cost

Reliability features

Feature	Protects against
Three triggers (startup, schedule, manual)	Missing new work while the laptop was off; easy testing
Skip already sent (dedupe on assignmentId)	Getting the same study guide every run
Retry On Fail on network/AI nodes	One-off hiccups from Moodle or Gemini
Fallback in Split chosen	Gemini answering with something that isn't a usable list
Error workflow: Error Trigger → Gmail	Silent failures: expired token, Gemini down, an assignment with no PDF. Any failed automatic run emails me the node and the error message.

Security habits

Habit	Why
Secrets only in n8n credentials	Encrypted at rest; never included when the workflow is exported as JSON
Never paste tokens into chats or screenshots	A Moodle token equals app-level access to the account; a leaked one is reset immediately
Clipboard-only token handling	The PowerShell helper never prints the token
Private token ignored	It can create website auto-login links, so it was never stored
Human review	Placeholders, [VERIFY] tags and the email reminder force a real review before anything is used

Cost: zero, and provably so

- **n8n**: self-hosted community edition, free (plus a free license key for extra debugging features).
- **Gemini**: free-tier API key from AI Studio. A Gemini app subscription does not include API access; the free tier covers this volume. Over the limit means a 429 error, not a bill.
- **Google Cloud** (OAuth for Drive and Gmail): no billing needed. The billing account Google prompted for was never activated (it wanted a $30 prepayment), then closed with no projects linked. Nothing can be charged.

Google sign-in detail

Self-hosted n8n can't use n8n's managed Google login, so I created my own Google Cloud project: enabled the Drive and Gmail APIs, set up an External consent screen with myself as a test user, and created a Web OAuth client with the redirect URI `http://localhost:5678/rest/oauth2-credential/callback`. In **Testing** mode Google expires the sign-in after 7 days; switching the app to 'In production' (still unverified, for personal use) removes that.

PART 7

Limitations, roadmap and FAQ

Known limitation	Planned fix
Two new assignments in one run get mixed (several nodes use .first())	Loop Over Items after 'Skip already sent'
Assignment with instructions but no attached file	Branch: send the instruction text to Gemini (for now the error alert catches it)
Dedupe happens before processing, so a failed run won't retry that assignment	Error alert tells me; could move dedupe to the end
Only PDF lecture files are used	Convert .pptx/.docx via Drive (convert=true → export PDF)
Flash-Lite is shallow	Per-question calls or a stronger model with a longer timeout
Intermediate Google Docs pile up in Drive	Delete them after export
Personal settings are inside nodes	A 'Settings' node (email, URL, model) so a classmate can import and plug in their own credentials

Is it allowed? It uses the API the university enabled for the mobile app, with my own account, at a low request rate. It's visible as API usage, not hidden. The final word is the university's IT acceptable-use policy.

Does it cost anything? No. Self-hosted n8n, free-tier Gemini, and Google APIs with no billing account.

Does it do the homework? It produces a study guide that teaches the concepts and points to the exact lecture pages. Anything needing real sources, team decisions or judgment is flagged, and the thinking and the submission stay mine.

Can I have it? Yes. Import the workflow JSON into your own n8n (Docker, ~5 minutes), then create your own Moodle token, Gemini key and Google credentials. None of my secrets are in the file.

What did building it teach? Docker, REST APIs and token auth, SSO-aware login flows, OAuth2, n8n expressions and binary data, prompt design, a retrieve-then-generate AI pipeline, and error handling, all on a real system.